IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 19 August 2024, accepted 3 October 2024, date of publication 11 October 2024, date of current version 8 November 2024.

Digital Object Identifier 10.1109/ACCESS.2024.3478831

APPLIED RESEARCH

A Comprehensive Design of Hybrid Residual (2+1)-Dimensional CNN and Dense Networks With Multi-Modal Sensor for Fish Appetite Detection

INFALL SYAFALNI[1,2,3], (Member, IEEE), AGAPE D'SKY[1], (Graduate Student Member, IEEE), NANA SUTISNA[1,2], (Member, IEEE), AND TRIO ADIONO[1,2], (Senior Member, IEEE)

[1]School of Electrical Engineering and Informatics, Institut Teknologi Bandung, Bandung 40132, Indonesia
[2]University Center of Excellence on Microelectronics, Institut Teknologi Bandung, Bandung 40132, Indonesia
[3]Systems Design Lab, School of Engineering, The University of Tokyo, Tokyo 113-8656, Japan

Corresponding author: Infall Syafalni (infall@ieee.org)

This work was supported by the Young Researcher Institut Teknologi Bandung (ITB) Program 2024 under Grant 959/IT1.B07.1/TA.00/2024.

ABSTRACT Fish is one of the most demanding protein sources in the food industry. However, the increasing demand must be followed by increasing production efficiency. One of the problems in fish production efficiency is an ineffective feeding method. In this paper, we address the problem of fish feeders using artificial intelligence in an aquarium. We propose fish appetite detection using multi-modal sensors resulting in the data for our AI system. The main aim is to improve the efficiency of fish feeding using a multi-modal sensor system. Our AI system consists of Residual (2+1)-Dimensional Convolutional Neural Networks (R(2+1)D-CNN) and dense networks to process video and accelerometer data. The video data is split into 20 frames and processed by an R(2+1)D-CNN. Furthermore, the accelerometer data is used to train several networks such as 1-dimensional Convolutional Neural Networks (CNN), Gated Recurrent Networks (GRU), and Dense Networks or Artificial Neural Networks (ANN). The system is implemented in an aquarium with two sensors *i.e.,* a webcam camera and an accelerometer and a main board processing using Raspberry-Pi 4. Experimental results show that the proposed system outperforms other methods with validation accuracy up to 99.09% for the Zeromean dense model and up to 99.39% for the Filtered dense model. The results also show that the multi-modal sensor system increases the accuracy significantly without a significant increase in the number of parameters. The work is useful for automation and efficiency in aquaculture.

INDEX TERMS Multi-modal sensor, R(2+1)D-CNN, GRU, ANN, fish feeder.

I. INTRODUCTION

Fish is one of the top contributors to the food industry. As a great source of protein, fish production, especially aquaculture, has been necessary for fulfilling food demands in the past decades. However, maintaining production growth is one of the biggest concerns in the aquaculture sector. From the 1990s until the 2010s, global aquaculture production was increasing significantly. Based on the Food and Agriculture Organization of the United Nations (FAO) and National Oceanic and Atmospheric Administration US (NOAA) [1], [2], the aquaculture product almost doubles every decade. However, this trend does not continue in 2020 as the production growth rate starts to saturate. The total global aquaculture production only increased from 71.5 million tonnes per year in 2010 to 87.5 millions tonnes per year in 2020. Another problem comes from the consumption rate. In 2023's FAO Agricultural Outlook [1], the global per

The associate editor coordinating the review of this manuscript and approving it for publication was Zhengqing Yun.

capita fish consumption rate is expected to increase from 20.4 kg/year to 21.2 kg/year. This implies that more efforts are needed to be done to accelerate aquaculture production.

The introduction of technology-aided systems has been able to improve production fascinatingly. The emergence of technology helps farmers to increase their profit by minimizing operational costs and increasing production efficiency. These technologies include Artificial Intelligence (AI) [3], [4], Internet of Things (IoT) [5], and automation [6], [7], [8]. Nevertheless, there hardly are any explorations around the use of multiple kinds of data in a single machine learning model. Thus, this paper discusses the first step of utilizing multi-modal AI models to improve aquaculture efficacy. In this case, we develop an intelligent system for a fish feeder in an aquarium using multi-modal sensors with hybrid Residual (2+1) Dimensional Convolutional Neural Networks (R(2+1)D-CNN) and Artificial Neural Networks (ANN) or dense networks.

Various research works have been done recently to apply technological support in aquaculture. These researches aim to tackle several major problems in aquaculture such as monitoring water parameters using IoT systems [9], [10], [11], [12], over or under fish feeding [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], and fish behavior analysis [8], [9], [25], [26]. The application itself varies along several constraints, such as hardware resources, implementation scale, different subjects, and environments.

One of the problems in aquaculture production is caused by feeding inefficiency. In aquaculture, feeding inefficiency is the biggest issue regarding the production yield. Feeding operation takes the largest portion of an aquaculture budget. Based on FAO and NOAA [1], [2], feeding spends 60-70% of the total budget in general. However, a huge portion of the fish feed gets wasted, further polluting the water with ammonia. In some cases, this may accumulate and cause mass death to the fish in the pond.

Moreover, many aquaculture operations are undergoing several inefficient processes due to manual procedures. The fish farmers usually make use of their senses and intuition to determine the right feeding time and portion. Additionally, they typically tend to overfeed the fish to guarantee the fish get everything they need. This shows that aquaculture production efficiency can be lifted with proper management above the feeding process. For this reason, technology has been partially used to overcome this problem.

In this paper, we address the problem of a fish feeder using artificial intelligence in an aquarium. We proposed a fish appetite detection using multi-modal sensors resulting in the data for our AI system. Our AI system consists of R(2+1)D convolutional layers and dense networks to process video and accelerometer data. The video data is split into 20 frames and processed by an R(2+1)D convolutional neural networks. The accelerometer data is used to train a neural network. The overall illustration of the proposed system using hybrid multi-modal sensors is shown in Figure 1. Moreover, we summarize our main contributions as follows:

1) We introduce a multi-modal sensors deep learning system for an aquaculture application.
2) The proposed method uses R(2+1)D-CNN layers to process the video data and a neural network to process the accelerometer data. The two outputs are concatenated in a fully connected layer.
3) The system is implemented in an aquarium with two sensors *i.e.,* a webcam camera and an accelerometer and a main board processing using Raspberry-Pi 4.
4) Experimental results show that the proposed system outperforms other methods with an accuracy of 99.09% for the Zeromean dense model and up to 99.36% for the Filtered dense model. The work is useful for automation and efficiency in aquaculture.

The organization of the paper is as follows: Section I explains the background of the work. Moreover, Section II describes the related works and the properties of the research. Next, Section III explains the fundamental building blocks for the proposed method consisting of the proposed method illustration, deep learning methodology, building block fundamentals, artificial neural networks, recurrent neural networks, convolution neural networks, and hyperparameter tuning. Section IV explains the details of the proposed method; fish appetite detection using hybrid R(2+1)D-CNN and dense networks. Finally, Section V shows the experimental results and Section VI concludes the paper.

II. PRELIMINARIES

A. RELATED WORKS

One of the research topics of present-day aquaculture advancement is the fish monitoring system. The work [11] focuses on real-time monitoring and predictive analysis of water parameters to enhance aquaculture practices. Moreover, the work [10] presents an aquaculture monitoring system employing NB-IoT technology for controlling water quality parameters through sensor and actuator integration. The work [25] explores AI techniques for detecting anomalous fish behaviors underwater, enabling early intervention measures. The work [4] describes the design of a smart seawater aquarium system leveraging AI technology to reduce stress levels and enhance the survivability of marine pets. The work [27] discusses fish monitoring in aquaculture using multibeam echo sounders and machine learning algorithms. The work [28] presents a method for detecting fish shoal behaviors utilizing convolutional neural networks and spatiotemporal information. The work [29] created an integrated sonar and camera system to monitor the fish's behavior using direct observation. This method excels against image intensity problems since the sonar can reconstruct underwater images even in total darkness. An advanced study in [30] also suggests that individuality-based aquaculture by assessing fish using iris biometrics can be done in the future.

As for the research in aquaculture feeding control, the work [18] developed an intelligent eel-feeding controller using photoelectric sensors. The feeding times are preset in

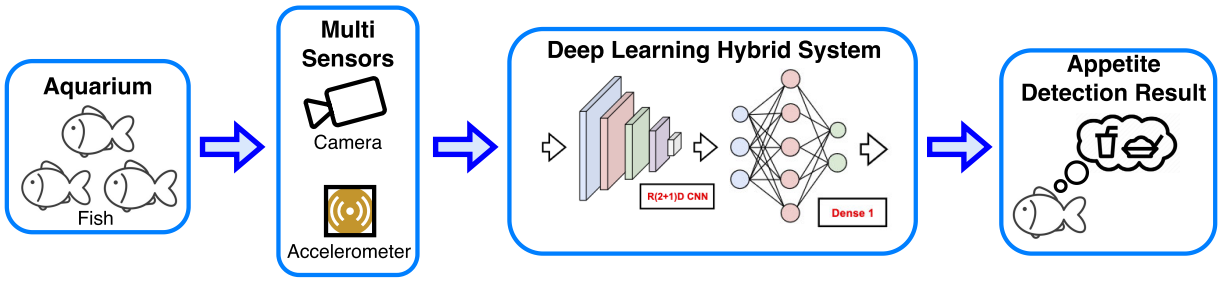


FIGURE 1. Deep learning hybrid system illustration.

the system. The work [19] showed that an accelerometer can be used to detect feeding activity if the data is taken with proper settings to reduce the amount of unnecessary information. The accelerometer detects the movement created by the water surface's ripples caused by the fish. This concept is advantageous with the use of floating pellets. The work [26] develops a pinger device, also using acceleration data, to classify fish feeding behavior. The work [20] built a system to identify fish's spontaneous collective behavior in a Recirculating Aquaculture System (RAS) to assess their appetite. The idea is that upon a feeding stimulus, the fish may change their motion patterns and move toward the feeding spot. This information can be obtained by observing the shedding light on swimming patterns, dispersion degree, interaction forces, and water flow fields. The work [21] proved that this system increases the feeding efficiency significantly by lowering the amount of residual feed. This method, however, does not work well with conditions like turbid water, and blending specimen color.

Furthermore, the use of neural networks for feeding control has seemed to be more popular recently. Some researches focus mainly on visual AI, delving deeper into feeding intensity assessment and leveraging neural networks to optimize feed dispensing based on fish behavior data [15], [16]. The work [22] addressed the feeding problem using a system to detect underwater pellets. The system utilizes visual object detection to spot and count the uneaten fish pellets. This method works great with the use of sinking pellets. For this specific environment, the water must have reduced turbidity. The work [22] developed a computer-vision-based intelligent fish-feeding system for black porgy using videos as its input. The system is also equipped with several water quality sensors to aid the feeding decision-making. This method is highly reliable for floating pellets, even when the water is turbid. Similar to this one, the work [8] introduces an automatic fish-feeding machine controlled by computer vision, which extracts features from both nutriment and ripple behavior to regulate feeding. The work [31] also proposes automated feeding control with multi-detection for dense fish tanks, utilizing computer vision to detect feeding activity and manage feed distribution. The work [13] employs image processing and machine learning to classify hunger levels in fish using an underwater camera placed at the bottom of the pond. The work [32] introduces an intelligent behavior-based fish-feeding system utilizing computer vision and fuzzy logic algorithms. The work [20] implemented the ANFIS system (Adaptive Neural Fuzzy Inference System) to control the feeding decision by assessing water quality parameters (dissolved oxygen and temperature). This model successfully suppresses ammonia content inside the water and gives minor improvement to the fish's final weight, since the temperature is highly correlated with the fish's life quality [33]. The work [14] develops an adaptive neural-based fuzzy inference system (ANFIS) for feeding decision-making in silver perch culture, specifically focusing on dissolved oxygen levels.

B. AQUACULTURE TECHNOLOGIES

In aquaculture, fish welfare is the most dominant aspect of all, as it directly controls the resulting product. Every other scope of an aquaculture system, such as environmental issues, health, and diseases, is also positively correlated to fish welfare [34]. Current findings also proved that fish can experience physical pain and mental stress [35], which increases the concern of a more humane and subject-centric aquaculture industry. FAWC [36] defined fish welfare regarding these 5 indicators: freedom from hunger and thirst, freedom from discomfort, freedom from pain, disease, or injury, freedom to express normal behavior, and freedom from fear and distress.

Incorporating AI into aquaculture technologies has significantly expanded the scope and capabilities of fish welfare management. Through leveraging AI algorithms and data analytics, we can gain deeper insights into fish behavior, environmental conditions, and overall system performance. For example, AI-powered monitoring systems can analyze vast amounts of data from sensors and cameras in real-time, detecting early signs of stress, disease, or abnormal behavior patterns in fish populations [29], [37], [38], [39]. Moreover, AI-driven predictive analytics can forecast potential welfare issues based on historical data trends, allowing preventive measures before problems escalate. Additionally, AI-based decision support systems can optimize feeding schedules and water quality management protocols, taking into account various factors such as species-specific nutritional

requirements [8], environmental parameters [40], and feed conversion rates [20]. The integration of AI technologies holds immense promise for advancing fish welfare standards in aquaculture, facilitating more sustainable and ethically responsible practices in the industry [3], [4], [21], [40].

C. SIGNAL PROCESSING IN AQUACULTURE

Identification of fish welfare is not simple since there is no straight method to measure the fish's hunger, stress level, pain, fear, and abnormal behavior. Workarounds have been taken to figure out indirect yet powerful ways to quantify these things efficiently.

The work [41] presents some of the explored methods for aiding the aquaculture industry, especially to support the fish hunger quantization issue. The fish appetite can be assessed using image and video observations [18], [29], [37], [39] (camera, IR sensors), acoustic technologies [27], [29] (sonar, acoustic tag, hydrophone), movement detection [16], [26] (accelerometer, magnetometer, gyroscope), environmental conditions [6], [20], [42] (dissolved oxygen, temperature, and other sensors), or even biological indicators [43], [44], [45] (EMG). These methods highly rely on data processing approaches.

In general, the two main strategies to execute data processing are using deterministic and stochastic means [46], [47], [48], [49], [50]. Deterministic solutions are often applied to data with feature(s) that can be inferred using specific procedures [46], [47]. In this way, the output characteristics can be derived directly from the models. Signal processing is a part of this method. On the other hand, stochastic solutions, such as learning models, involve a certain degree of randomness to solve a problem [48], [49], [50]. These two ways are integral to the recent development in aquaculture technologies.

1) REAL-TIME DATA PROCESSING

Newly designed hardware and algorithms enable systems to gain high throughput. Using these technologies, crucial aquaculture real-time applications, such as fish monitoring systems, and environment parameter control, can be carried out better than before, namely by reducing the computation latency and increasing the overall computational speed. In aquaculture systems, where immediate response to changes in fish behavior or environmental conditions is critical, ensuring real-time processing is essential [6], [51], [52], [53].

2) ENHANCED RELIABILITY

In aquaculture operations, where continuous monitoring and control are necessary for maintaining optimal conditions, reliability is paramount. A higher degree of reliability itself is achieved when a system is robust, less prone to errors, and has an adequate level of precision. Recent technologies are now able to work under multiple environments and constraints with improved decision accuracy [40], [54].

3) SCALABILITY AND FLEXIBILITY

Recent research in aquaculture involves learning models, especially the ones with the aid of artificial intelligence. These learning models often be used in different use cases, depending on the fish as the subject, its behavior, as well its environment, by changing the models' parameters. This is why reusing hardware and algorithms in AI is quite common, thus increasing the flexibility for implementation and scalability for mass production purposes [7], [40], [55].

III. FUNDAMENTAL BUILDING BLOCKS FOR PROPOSED SYSTEM

In this section, the fundamental blocks for building the proposed method are explained. The parts are the proposed system illustration to give a brief description of the proposed method, the deep learning concept, the building block fundamentals (neuron calculation, activation functions, and pooling algorithms), artificial neural networks (ANN), recurrent neural networks (RNN), convolutional neural networks (CNN), and hyperparameter tuning.

A. PROPOSED SYSTEM ILLUSTRATION

In this work, we use the multi-modal sensor for the appetite detection of the fish. Figure 2 shows the overall proposed system. Two sensors, a camera and an accelerometer, are used as the input of the detection main single board controller (SBC). In this case, we use a Raspberry Pi 4 for running the deep learning algorithms and controlling the fish feeder. Finally, the server is used for data cleansing, model validation, and model training.

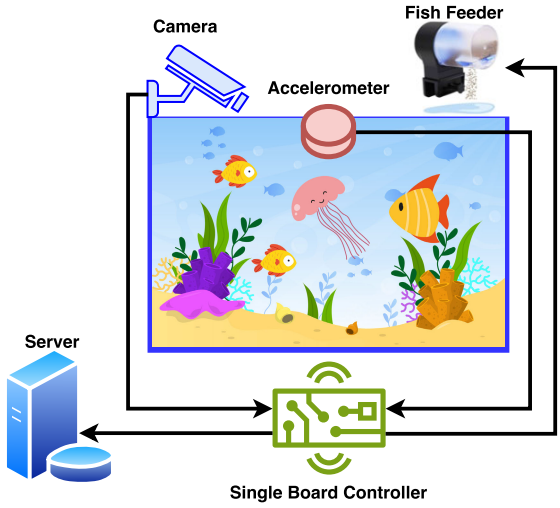


FIGURE 2. Illustration of the proposed system.

B. DEEP LEARNING

Deep learning is one of the most used methods for high-complexity tasks. As a subset of artificial intelligence, this method lets the machine solve nonlinear problems using

an iterative training process. Figure 1 shows the proposed system; fish appetite detection using hybrid multi-modal sensors. Figure 3 shows the flowchart of the proposed system.

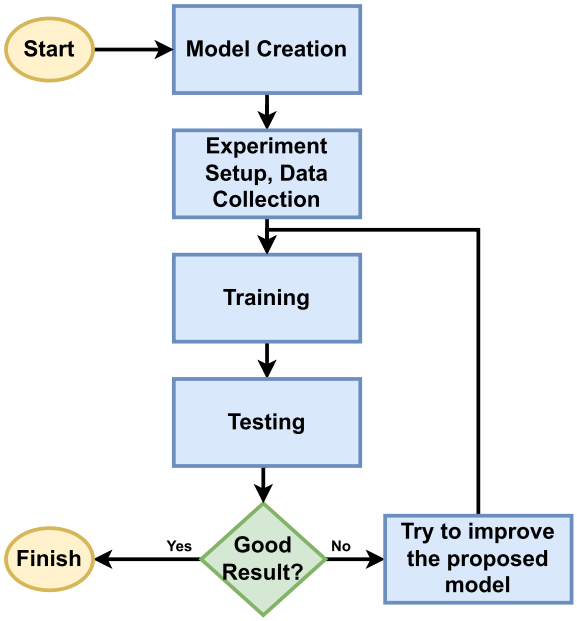


FIGURE 3. Flowchart of deep learning model development.

To deploy a deep learning method on a specific case, one must create a model, train it, and then validate it with real-life datasets as shown in Figure 3. A model can be seen as a black box, where the parameters inside it are not specified at first (random values). We can feed the model with various amounts of data so that the model can give us its perception (inference or forward propagation). The model's perception may differ from ours at first, and the quantization of this difference gives us error values. The error values can be relayed back to the model, allowing the model to fit itself to the provided data (backward propagation). This set of procedures is called the training process. After the training, we can validate the model by redoing the inference to a different set of data. Every block of a model has its algorithms to execute forward and backward propagation. Moreover, several blocks can be combined to create a deeper model with a higher level of complexity. These building blocks, particularly the ones that are used in the experiment, are described in the next subsection.

C. BUILDING BLOCK FUNDAMENTALS

Neural networks can be defined with 3 basic layers: **the input layer**, **the hidden layer**, and **the output layer**.

Definition 1: The input layer is defined as the first layer that consists of a neural network that receives the raw data for the model.

The input layer consists of values that we feed into the model, *e.g.,* sensor data, plot values, and trends. The number of neurons in the input layer depends on the number of inputs.

Definition 2: The output layer is defined as the last layer that consists of a neural network that represents the results of the deep learning model.

The output layer is composed of the desired values as the outcome of the model. The number of neurons in the layer depends on the task. For example, a detection task may have a neuron that outputs the probability of the detection. In the classification task, it may have more neurons to represent the probability of all classes.

Definition 3: Hidden layer(s) is defined as the layer(s) between the input and the output layers. The hidden layer(s) aims to learn patterns in the data. The hidden layer may have more than one neural network layer.

The hidden layer is the place where some predefined manner of computations exists, which turns the input values into the output values. Figure 4 describes the big picture of the neural networks.

The hidden layer composition generally is the heart of a model. The layer is structured in such a way that the model can learn patterns and make decisions effectively. To obtain these characteristics, the hidden layer may be built from several elementary operations, functions, and alterations. Various amounts of elementary blocks can be chosen based on the data and its core features, including basic math operations (typically multiply and accumulate), activation functions, pooling algorithms, and regularization algorithms.

1) MULTIPLY AND ACCUMULATE

Inside the neuron, there are multiply and accumulate (MAC) and activation functions to perform the function. Multiplication and addition are the two most common operations in neural networks, which conjointly create linear functions. Linear functions are crucial in this topic because the processing elements inside hidden layers need to take several input data to make their decisions. This will be shown in the next subsections through frequently used mathematical formulas.

Definition 4: Let x_i be the input value, w_i be the weight and b_i be the bias with index i. Multiply and accumulate is represented by $y_i = x_i w_i + b_i$.

By using the MAC module, an operation in a neuron called weighted sum can be calculated using Lemma 1.

Lemma 1: Let y_i be the weighted input with index i. The sum of the multiply and accumulate in the neuron, which we call the weighted sum, can be represented as $y = \sum_{i=0}^{N-1} y_i$. Note that when $b_i = b$ has only a value, the weighted sum becomes $y = Nb + (\sum_{i=0}^{N} x_i w_i)$, where N is the number of inputs.

2) ACTIVATION FUNCTIONS

Activation functions are one of the main aspects that differentiate networks in machine learning with linear combinations. Complex problems oftentimes cannot be solved with only linearly growing functions, so activation functions are introduced here to give some degree of nonlinearity.

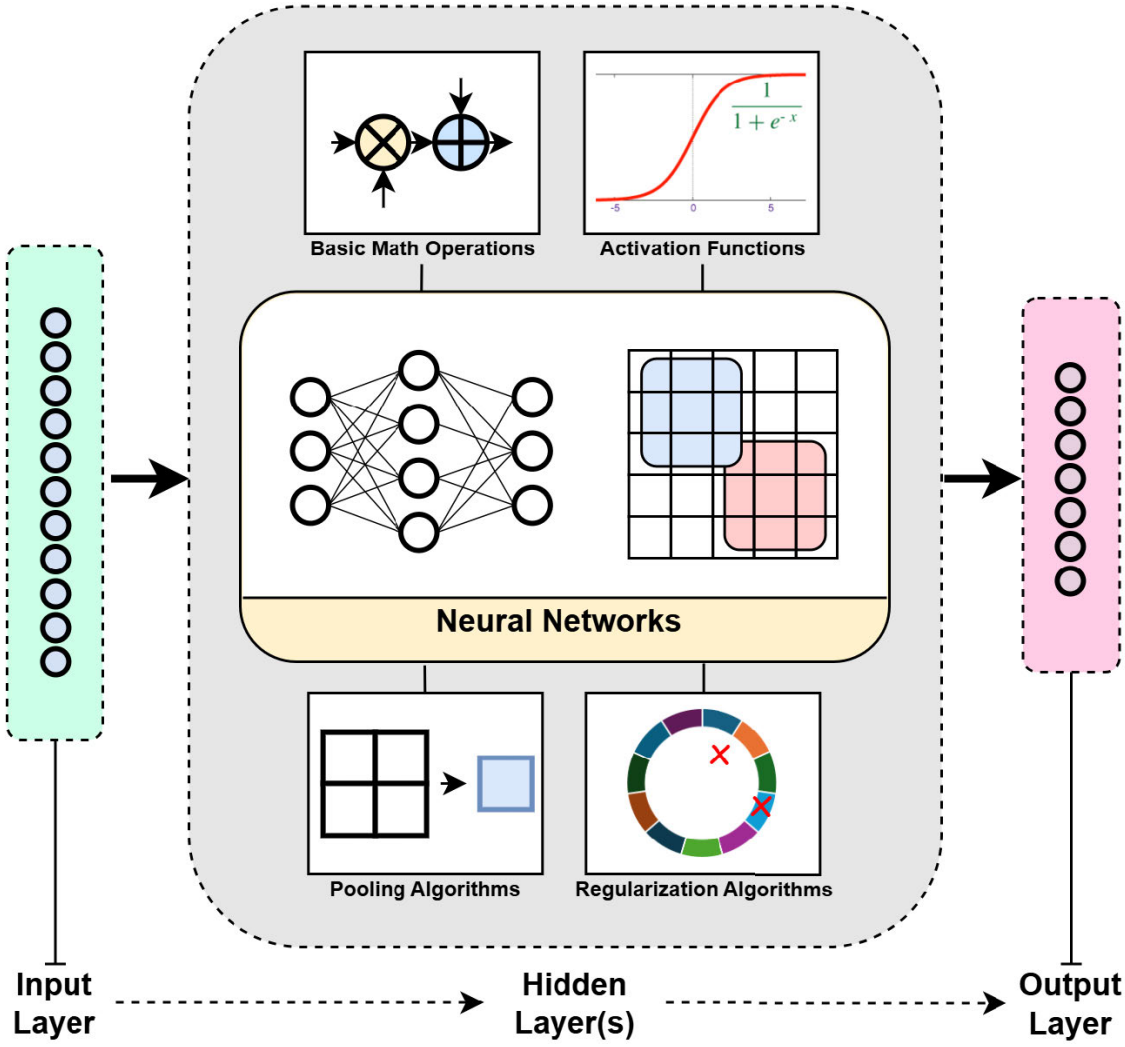


FIGURE 4. Basic building blocks in deep learning architecture.

Definition 5: An activation function is defined as a mapping of the weighted sum to a nonlinear function. The function determines whether a neuron is active or not for the corresponding inputs. Some of activation functions $f(y)$ are [56]:

- *1) ReLu;*

$$f(y) = max(0, y),$$

- *2) Leaky Relu;*

$$f(y) = \begin{cases} ax & for\ y < 0 \\ x & for\ y \geq 0, \end{cases}$$

- *3) sigmoid;*

$$f(y) = \frac{1}{1 + e^{(-y)}},$$

- *and 4) tanh;*

$$f(y) = tanh(y) = \frac{1 - e^{-2y}}{1 + e^{2y}},$$

where y is the input of the activation function in the form of a weighted sum.

Various types of activation functions can be selected based on the model's structure and data characteristics, depicted in Figure 5. Choosing one of these itself is quite a challenge, as the resulting accuracy level will hardly depend on this thing. Some functions, however, are preferred in some textbook applications. For example, the sigmoid and hyperbolic tangent functions are more commonly used in the finalizing layers.

3) POOLING ALGORITHMS

To create decisions, a large number of data needs to be reduced, and this is where pooling algorithms perform.

Definition 6: Pooling decreases the resolution of the inputs. Here are several pooling methods. The max-pooling is represented by

$$\rho = \max(x_i | i = 0, \cdots, N - 1),$$

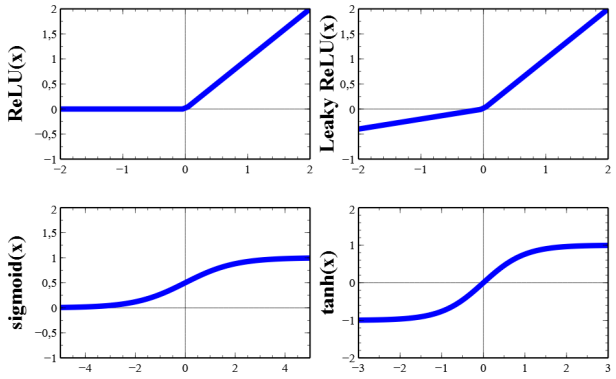


FIGURE 5. Activation functions: ReLU, Leaky ReLu, sigmoid, and tanh.

the median pooling is represented by

$$\rho = med(x_i | i = 0, \cdots, N-1),$$

and, the average pooling is represented by

$$\rho = \frac{\sum_{i=0}^{N-1} x_i}{N},$$

where $X = [x_0, x_1, \cdots, x_{N-1}]$ *is an input array with* N *elements, the operator* max *searches the maximum value of the inputs, the operator med searches the median value of the inputs, and* ρ *is the pooling output.*

Pooling layers decrease the number of information based on a specific algorithm, *e.g.,* maximum, average, and median (as Definition 6), thereby reducing the model's complexity while still maintaining a moderate level of overfitting possibilities. These algorithms are more commonly used in data with higher dimensionality.

4) REGULARIZATION ALGORITHMS

Regularization in machine learning puts a limit on how specific and specialized a model can become. It stops the model from trying too hard to match every tiny detail in the training data (to overfit). Instead, it helps the model focus on the big picture, enabling it to perform better for new real-world data.

Definition 7: Regularization prevents overfitting by some methods such as observing the loss function by giving a penalty to the loss function based on the weight changes and dropping out random neurons in the training phases.

Additionally, regularization also stops models from giving too much importance to outlier features, as it helps models stay steady even if we change some parts of the data slightly.

Lemma 2: Let N *be the number of weights and* α *is the learning rate for the neural network. L1, L2, and elastic regularizations are represented by the update of the loss function [56]:*

$$L' = L + \zeta,$$

where L' *is the total loss function,* L *is the original loss function, and* ζ *is* $(\alpha \sum_{i=0}^{N} |w_i|)$ *for L1,* $(\alpha \sum_{i=0}^{N} w_i^2)$ *for L2, and* $(\alpha_1 \sum_{i=0}^{N} |w_i| + \alpha_2 \sum_{i=0}^{N} w_i^2)$ *for elastic regularization.*

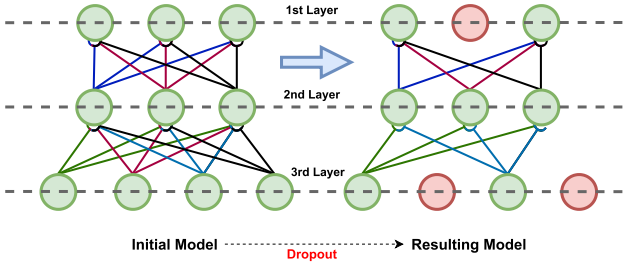


FIGURE 6. Layer dropout illustration for regularization.

Regularization is especially helpful when we don't have a lot of data or when the data is noisy. One of the simplest forms of regularization is to randomly neglect a few nodes in the model. Figure 6 shows that in the 1st layer, a neuron is dropped (red neuron) while in the 3rd layer, two neurons are dropped (red neurons).

D. ARTIFICIAL NEURAL NETWORKS

An Artificial Neural Network (ANN) is one form of the simplest model for deep learning. Every information that is fed to this model as an input can be seen as a node. A group of this information is called the input layer. Every node inside the input layer is connected to every node inside the following layer (so-called the hidden layer) and so on until it reaches the output layer. Figure 7 illustrates a clearer construction of ANN models.

For a better understanding of the neural network, we represent the neural network in the form of matrix operations using Lemmas 3 and 4. Figure 8 shows the example of multi-layer neural networks. In this case, we focus on the first and the second layers. The inputs come from the first layer while the outputs are resulted from the second layer.

Lemma 3: Suppose that M *is the number of neurons and* N *is the number of inputs for a neuron. The input matrix is*

$$X = \begin{bmatrix} x_0, & x_1, & \cdots & x_{N-1} \end{bmatrix},$$

and the weight matrix is

$$W = \begin{bmatrix} w_{0,0}, & w_{0,1}, & \cdots & w_{0,M-1}, \\ w_{1,0}, & w_{1,1}, & \cdots & w_{1,M-1}, \\ \cdots & \cdots & \cdots & \cdots \\ w_{N-1,0}, & w_{N-1,1}, & \cdots & w_{N-1,M-1} \end{bmatrix}.$$

The weighted sum matrix Y *can be represented as*:

$$Y = X \times W + B,$$

where $Y = [y_0, y_1, \cdots, y_{M-1}]$ *is the weighted sum outputs (matrix) for* M *neurons and* $B = [b_0, b_1, \cdots, b_{M-1}]$ *is the bias vectors.*

Lemma 4: Suppose that the neuron has only a bias value $b_i = b$ *for all the products. Thus by Lemma 1, the input matrix* X *and the weight matrix* W *can be modified as follows*

$$X' = \begin{bmatrix} x_0, & x_1, & \cdots & x_{N-1}, & N \end{bmatrix},$$

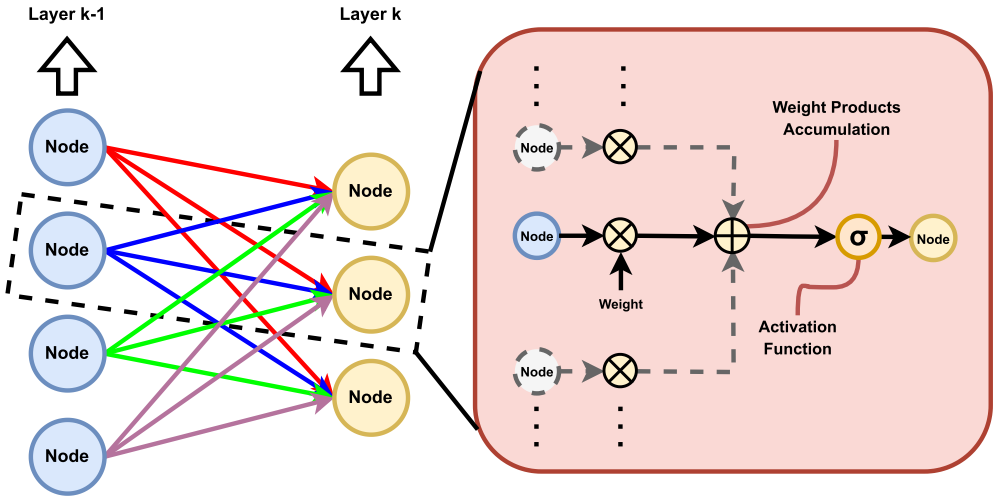


FIGURE 7. Detail artificial neural network illustration.

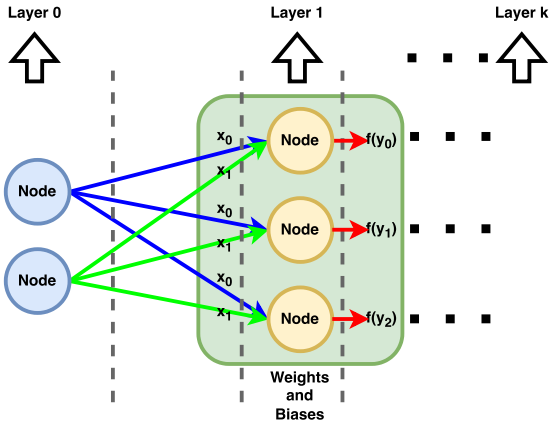


FIGURE 8. Example for neural networks calculation.

and

$$W' = \begin{bmatrix} w_{0,0}, & w_{0,1}, & \cdots & w_{0,M-1}, \\ w_{1,0}, & w_{1,1}, & \cdots & w_{1,M-1}, \\ \cdots & \cdots & \cdots & \cdots \\ w_{N-1,0}, & w_{N-1,1}, & \cdots & w_{N-1,M-1} \\ b_0, & b_1, & \cdots & b_{M-1} \end{bmatrix}.$$

The weighted sum matrix Y can be represented as:

$$Y = X' \times W',$$

where $Y = [y_0, y_1, \cdots, y_{M-1}]$ is the weighted sum outputs (matrix) for M neurons and $B = [b, b, \cdots, b]$ is the bias vectors.

Proof: The size of matrix X is N and the size of matrix W is $M \times N$. The value of bias b is required to be added in each column product summation. By adding one element in matrix X and a row of b values in matrix W, we have the lemma. □

Example 1: To calculate the outputs of the neural networks in Figure 8 using Lemma 4 and Definition 5, we have the modified input matrix as

$$X' = [x_0,\ x_1,\ 2].$$

Next, the modified weight matrix is

$$W' = \begin{bmatrix} w_{0,0}, & w_{0,1}, & w_{0,2}, \\ w_{1,0}, & w_{1,1}, & w_{1,2}, \\ b_0, & b_1, & b_2 \end{bmatrix}.$$

Thus, the weighted sum matrix is calculated by $Y = [y_0, y_1, y_2] = X' \times W'$. Finally, by the activation function, the output matrix is $f(Y)$. Suppose that we use a sigmoid function as the activation function. Thus, we have $f(y_0) = 1/1 + e^{(-y_0)}$, $f(y_1) = 1/1 + e^{(-y_1)}$, and $f(y_2) = 1/1 + e^{(-y_2)}$. ■

In the neural network, nodes are connected through edges. During the forward propagation, edges denote a linear process inside nodes, that is, a multiplication by a weight w and an addition by a bias value b (Definition 4). After that using Definition 5, a nonlinearity is applied using an activation function *e.g.,* rectified linear unit (ReLU), Leaky ReLu, sigmoid, and tanh. Therefore in Figure 7, the output of every node in layer k is $f(Y_k)$, where f is the activation function, Y_k is the weighted sum matrix calculated using Lemma 4, and N_k denoting the number of the inputs from layer $k-1$ entering the nodes.

Theorem 1: Suppose that we have K layers of neural networks. Thus, the output of the last layer (output layer) is represented by the following matrix operations:

$$\begin{aligned} f(Y_{K-1}) &= f(\cdots f(f(f(X' \times W'_0) \times W'_1) \cdots \times W'_{K-1}) \\ &= f(f(Y_{K-2}) \times W_{K-1}), \end{aligned}$$

where $X' = f(Y_{-1})$ is the modified input matrix with bias integration, W'_0 to W'_{K-1} are the modified weight matrices

with biases integration for K layers, f is an activation functions, and $f(Y_k)$ is the output of kth layer.

Proof: By Lemma 4, the modified of input matrix $X' = f(Y_{-1})$ consists of N_0 inputs with $N_0 + 1$ elements. In this case, we have the first layer output is $f(Y_0) = f(X' \times W_0')$. The modified weight matrix W_k consists of M_k arrays of weights, where each weight array consists of N_k weights and a bias. The matrix multiplication dimension in kth layer is $N_k + 1 \times (M_k, N_k + 1) \rightarrow M_k$, where N_k is the number of inputs and M_k is the number of neurons or outputs in the kth layer. Finally, the output of kth layer is $f(Y_k) = f(f(Y_{k-1}) \times W_k)$, where $k = 0, 1, \cdots, K - 1$. Thus, we have the theorem. □

Lemma 5: The backpropagation for updating the weight and the bias is represented by [56]:

$$w' = w - \alpha \frac{\partial f(y)}{\partial w}, \text{ and } b' = b - \alpha \frac{\partial f(y)}{\partial b},$$

where $\frac{\partial f(y)}{\partial w}$ and $\frac{\partial f(y)}{\partial b}$ are the derivative of the weight and the bias from the changes of the output $f(y)$, respectively, and α denotes the learning rate.

Proof: Let the desired value for the node be y and the output value be $y + \delta$. Then, we can get the error value for this particular node as a function of y and $y + \delta$. This function is called the cost function. Utilizing this function, we can update the parameters (w and b), so that the next inference will minimize the error using the equation

$$\begin{aligned} w' &= w - \alpha \lim_{\delta \to 0} \frac{f(y + \delta) - f(y)}{\delta} \\ &= w - \alpha \frac{\partial f(y)}{\partial w}, \end{aligned}$$

where α denotes the learning rate. The same calculation is applied for b. Thus, we have the lemma. □

E. RECURRENT NEURAL NETWORKS

A Recurrent Neural Network (RNN) is a type of network that processes sequential data, where the output of the timestamp t is affected by the result on the previous timestamp $t - 1$ as illustrated in Figure 9.

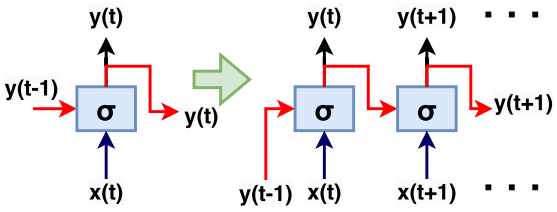


FIGURE 9. RNN illustration.

Lemma 6: The forward propagation formula can be written as [58]:

$$y_t = \sigma(v_t y_{t-1} + w_t x_t + b_t),$$

where x_t is the input value, σ is a sigmoid activation function, v_t and w_t are the previous output and current weight values, respectively, y_{t-1} is the previous output, and b_t is the bias value.

The key distinction between an RNN layer and an ANN layer is the additional term $v_t y_{t-1}$, with v_t as the previous output's weight value.

Lemma 7: As for the backward propagation in Lemma 5, using the partial derivative to a specific parameter, we update the weight by the following:

$$v' = v - \alpha \frac{\partial f(y)}{\partial v}, \tag{1}$$

where v' is the updated weight, $f(y)$ is the output value, and α is the learning rate.

The introduction of RNNs has unlocked a whole new aspect of artificial intelligence, as RNNs can perform better than ANNs on some time-dependent and sequential data, such as real-time signals, natural language models, etc. Still, simple RNN models may have problems with high-complexity data and features.

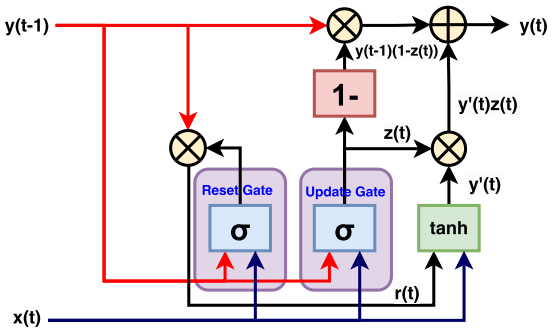


FIGURE 10. GRU-RNN illustration.

There are a few modifications that have been made to the RNN to solve problems existing in standard RNN, e.g. vanishing gradient, and one of them is the Gated Recurrent Unit (GRU). Instead of treating every piece of information in the sequence homogeneously, this model learns to update and reset and therefore can forget unnecessary data. The illustration is given in Figure 10.

Lemma 8: By using Lemma 3.6, the reset gate resulting in the reset value r_t and the update gate resulting in the update value z_t are represented by [58]:

$$r_t = \sigma(w_r x_t + v_r y_{t-1} + b_r),$$

and

$$z_t = \sigma(w_z x_t + v_z y_{t-1} + b_z).$$

These two paths are used concurrently to do the model's inference. Here, r_t represents the quantization of how much we need to ignore the previous result, whereas z_t represents how much we need to consider the new result computed in the same layer, y_t', formulated as

Lemma 9: The result for the GRU network is represented as [58]:

$$y_t' = \tanh(w_{y1} y_{t-1} r_t + w_{y2} x_t), \tag{2}$$

where tanh *is the activation function, and* w_{y1} *and* w_{y2} *are the additional weights.*

Lemma 10: At last, the final decision (the output) of this layer can be written as [58]:

$$y_t = \sigma(w_{y3}(y_{t-1}(1 - z_t) + y'_t z_t)), \tag{3}$$

where y'_t *is the new result value,* z_t *is the update value,* r_t *is the reset value, and* w_{y3} *is the final output weight.*

The backpropagation algorithm for GRU holds the same fundamental concept as the one introduced previously. We take the loss value and take its derivative to the parameters inside the model to get correction values. The only distinction lies in the loss formula, that is, it should be the summation of the losses from each timestamp. Also, the backpropagation is applied on the reset path and update path individually.

F. CONVOLUTIONAL NEURAL NETWORKS

A convolutional neural network (CNN) is a network whose computation utilizes discrete convolution algorithms. The algorithm itself can be varied for 1-dimensional, 2-dimensional, or even higher-dimensional cases, depending on the data used. A simple 1-dimensional CNN algorithm is depicted in Figure 11.

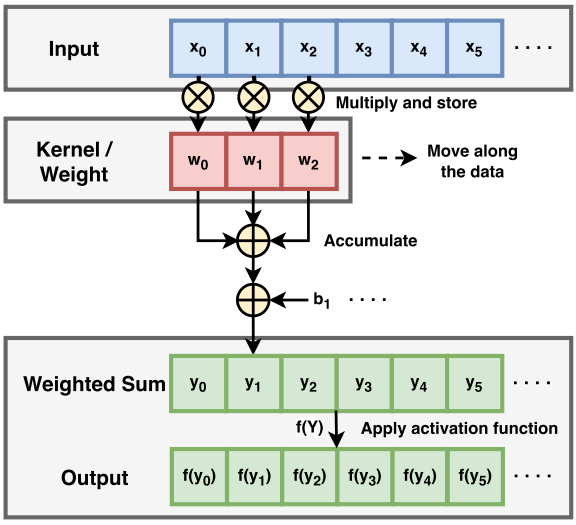


FIGURE 11. 1-Dimensional CNN illustration.

Lemma 11: The forward propagation for the 1-D CNN is defined using the equation

$$y = \sum_{i=0}^{N-1} x_i w_i + b_i, \text{ and } f(y) = \sigma(y),$$

where $f(y)$ *is the output,* x *is the input,* σ *is the sigmoid activation function,* w *is the values in the kernel, and* b *is the bias value. Note that the kernel size is* N *for a one-dimensional array.*

The backpropagation general idea and properties are still quite identical to the ones previously discussed in Lemma 5.

The addition of CNN dimensionality only affects the number of identical computations needed, whereas the approach persists. In a 2-D CNN layer, for example, the input, the output, and the kernel are all 2-dimensional data, and thus a nested summation is needed. Based on Figure 12, every output pixel value is derived from the same pixel's position in the input and its neighbors.

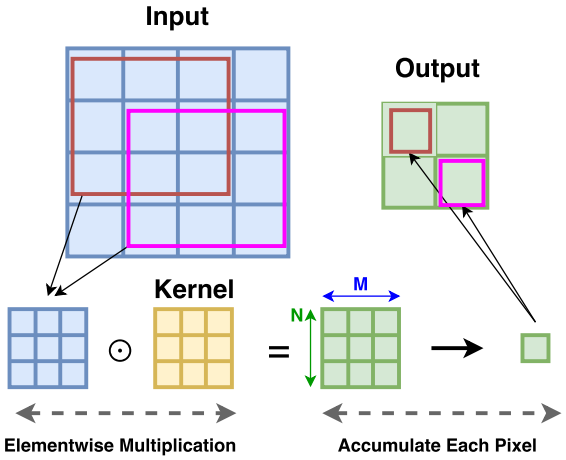


FIGURE 12. 2-Dimensional CNN illustration.

Lemma 12: For representing the 2-dimensional CNN, the output $f(y)$ *is represented by the following*

$$y = \sum_{j=0}^{N} \sum_{i=0}^{M} x_{i,j} w_{i,j} + b_i, \text{ and } f(y) = \sigma(y),$$

where $f(y)$ *is the output,* x *is the input,* σ *is the sigmoid activation function,* w *is the values in the kernel, and* b *is the bias value. Note that the kernel size id* $N \times M$ *for a two-dimensional matrix.*

G. HYPERPARAMETER TUNING

A model hyperparameter is a configuration that is external to the model and whose value cannot be estimated from data. Every model contains hyperparameters. Different models may have different hyperparameters. These parameters need to be configured to achieve the best result possible. Furthermore, different use cases of a model may need different model parameter values. This is why hyperparameter tuning is important, especially for deep learning.

An example can be derived from the ANN (Dense) model. Before we start the training process, we must first specify the model's characteristics through its hyperparameters, such as the number of layers, the number of nodes, activation functions, learning rate, etc. Setting these values properly will grant us better results. Sometimes, the best hyperparameters cannot be defined analytically, thus multiple trainings must be done to determine the most suitable parameters.

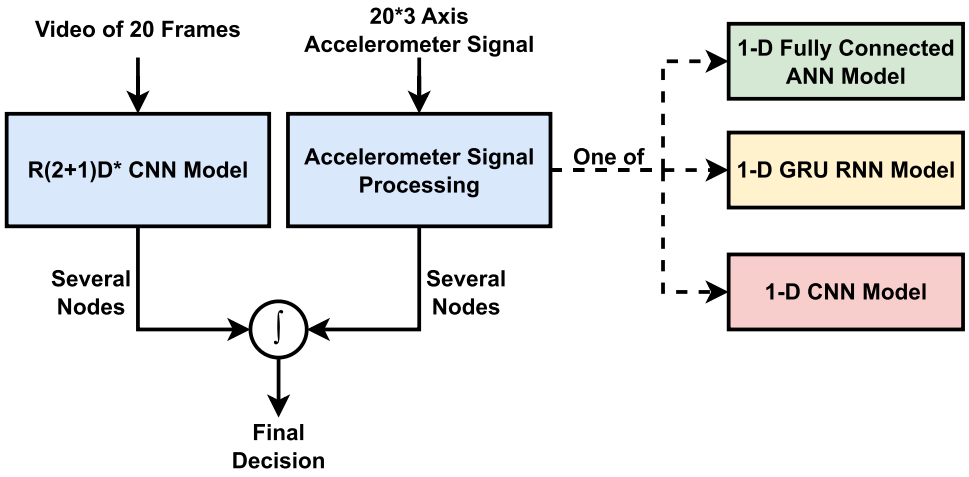


FIGURE 13. High-level abstraction for the proposed model.

IV. PROPOSED DEEP LEARNING ARCHITECTURE: HYBRID R(2+1)D-CNN AND DENSE NETWORK WITH MULTI-MODAL SENSORS

In this section, we explain the proposed method in detail as shown in Figure 13. We use multi-sensors for detecting the fish appetite in an aquarium. The sensors are a camera that captures video data and an accelerometer that captures the movement of the water surface. After that, the video data is processed using the R(2+1)D-CNN while the accelerometer data is processed using one of one dimensional ANN, GRU, or CNN. The outputs of both processed data then are concatenated and processed using the dense network to output the final decision. The more detailed design and analysis are explained in the following subsections.

A. R(2+1)D CONVOLUTIONAL NEURAL NETWORKS

Several variations exist for higher-dimension CNN algorithms. The work [22] showed that the 3D CNN algorithm can be broken down into 2D CNNs (spatial convolution) + 1D CNNs (temporal convolution), thus called the R(2+1)D-CNN Figure 14. With the proper way of tuning, this approach may increase the layer's complexity while conserving the number of its parameters. The work [22] used this model with fewer elementary layers to detect fish appetite and referred to this model as the R(2+1)D* CNN.

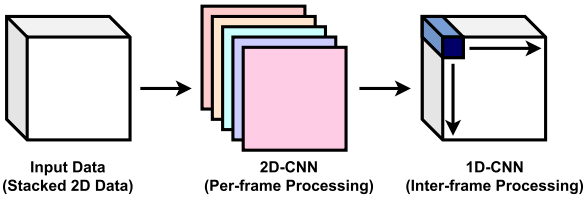


FIGURE 14. 3-Dimensional CNN with R(2+1)D concept.

Theorem 2: From Lemmas 11 and 12, the R(2+1)D CNN is represented by the following:

$$y = \sum_{k=0}^{R}\sum_{j=0}^{N}\sum_{i=0}^{M} x_{i,j}w_{i,j}w_k + w'_k + b'_k,$$
$$\text{and } f(y) = \sigma(y),$$

where $x_{i,j}$ is the input, $w_{i,j}$ and w_k are the 2D and 1D kernel/weight values, respectively, b_i and b_k are the 2D and 1D bias values, respectively, and $f(y)$ the sigmoid activation function.

Proof: Substituting the output of Lemma 12 to the input of Lemma 11, we have the following:

$$y = \sum_{k=0}^{R}(\sum_{j=0}^{N}\sum_{i=0}^{M} x_{i,j}w_{i,j}w_k + b_i)w_k + b_k,$$

where $N \times M$ is the size of 2D kernel, and R is the size of 1D convolution data. By entering the 1D weight value w_k and the 1D bias value b_k to the loop i and j, we have the following:

$$y = \sum_{k=0}^{R}\sum_{j=0}^{N}\sum_{i=0}^{M} x_{i,j}w_{i,j}w_k + w_k b_i + \frac{b_k}{NM}.$$

Since $N \times M$ is a constant, we can assign $\frac{b_k}{NM}$ as b'_k. Next, since w_k and b_i are updated based on the backpropagation as Lemma 5, we can combine $w_k b_i$ to become w'_k. Thus, we have the theorem. □

The details of the size of inputs and kernels are shown in Table 1. The image input dimension is 300 × 300 pixels with 20 frames in RGB color (3 channels). For example, in the third row of the table, the 2D kernel size is $N = 7 \times N = 7$ while the 1D kernel size is $R = 3$.

B. FILTERING ACCELEROMETER DATA

Additionally, for the accelerometer data, we created a different acceleration dataset by applying a high pass filter to

TABLE 1. Main R(2+1)D-CNN model architecture.

Layer	Input Dim.	Output Dimension	Sublayer
Image Input	20*300*300*3	-	
Image Rescaling	20*300*300*3	20*300*300*3	
R(2+1)D* 1	20*300*300*3	20*100*100*32	CNN2D (kernel 1*7*7, stride [1,2,2]) CNN1D (kernel 3*1*1, stride [1,1,1])
R(2+1)D* 2	20*100*100*32	20*100*100*32	CNN2D (kernel 1*3*3, stride [1,1,1]) CNN1D (kernel 3*1*1, stride [1,1,1]) CNN2D (kernel 1*3*3, stride [1,1,1]) CNN1D (kernel 3*1*1, stride [1,1,1])
R(2+1)D* 3	20*100*100*32	10*50*50*64	CNN2D (kernel 1*3*3, stride [2,2,2]) CNN1D (kernel 3*1*1, stride [1,1,1]) CNN2D (kernel 1*3*3, stride [1,1,1]) CNN1D (kernel 3*1*1, stride [1,1,1])
R(2+1)D* 4	10*50*50*64	5*25*25*128	CNN2D (kernel 1*3*3, stride [2,2,2]) CNN1D (kernel 3*1*1, stride [1,1,1]) CNN2D (kernel 1*3*3, stride [1,1,1]) CNN1D (kernel 3*1*1, stride [1,1,1])
R(2+1)D* 5	5*25*25*128	3*12*12*256	CNN2D (kernel 1*3*3, stride [2,2,2]) CNN1D (kernel 3*1*1, stride [1,1,1]) CNN2D (kernel 1*3*3, stride [1,1,1]) CNN1D (kernel 3*1*1, stride [1,1,1])
Average Pooling	3*12*12*256	1*4*4*256	
Output Layer	1*4*4*256	2	

the zero-mean data to attenuate lower frequencies. The filters used in this context were generated made using MATLAB, that is, by simply returning the filter coefficients based on the frequency characteristics of the filters. In this work, we use the Chebyshev Type II filter.

Lemma 13: The output filter $Y(z)$ is represented as:

$$Y(z) = X(z)[b_1 + b_2z^{-1} + \ldots + b_{k+1}z^{-k}] \\ -Y(z)[a_2z^{-1} + \ldots + a_{k+1}z^{-k}].$$

where Y is the output, X is the input, b is the numerator coefficients, a is the denumerator coefficients, and k is the order of the filter. The variable z in this equation implies the use of Z-domain, and thus z^{-k} denotes the introduction of sequential delay k.

Proof: The coefficients of the filter are part of the discrete domain transfer function of the filter. The coefficients are expressed as

$$\frac{Y(z)}{X(z)} = \frac{b_1 + b_2z^{-1} + \ldots + b_{k+1}z^{-k}}{1 + a_2z^{-1} + \ldots + a_{k+1}z^{-k}},$$

To determine the filtered output data using these coefficients, the discrete transfer function can be cross-multiplied and then rearranged to get a function that yields $Y(z)$. □

Lemma 14: The discrete-time domain form of the filter is represented by the following:

$$y(n) = b_1x(n) + b_2x(n-1) + \ldots + b_{k+1}x(n-k) \\ -a_2y(n-1) + \ldots + a_1y(n-k),$$

where y and x denote the time domain representations of Y and X respectively. Here, n is used as the time variable.

C. BUILDING THE MODELS

Several methods can be exploited to assess fish's appetite. The fish's appetite can be distinguished from the water quality and the fish's collective behavior. The fish farmers often check the water quality using bare hands to predict the tendency of the fish to eat. A more direct approach is to directly observe the splashing and flocking behavior of the fish near the feeding spot at the usual feeding time. Here, we make use of the fish's collective behavior to determine the fish's appetite.

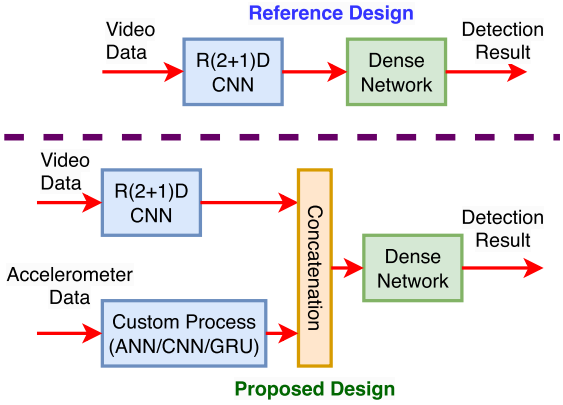


FIGURE 15. Comparison between the referenced model and proposed model.

1) REFERENCE MODEL

The work [22] showed that the utilization of R(2+1)D* CNN model for black porgy aquaculture pond offers a high level of accuracy with fewer parameters. We modified the model and made a few adjustments to check whether it suited our use case. Table 1 gives the detailed adjusted model's architecture. The model takes videos (20 frames of 300*300 pixels, RGB) as its input, computes the data using 5 layers of R(2+1)D*

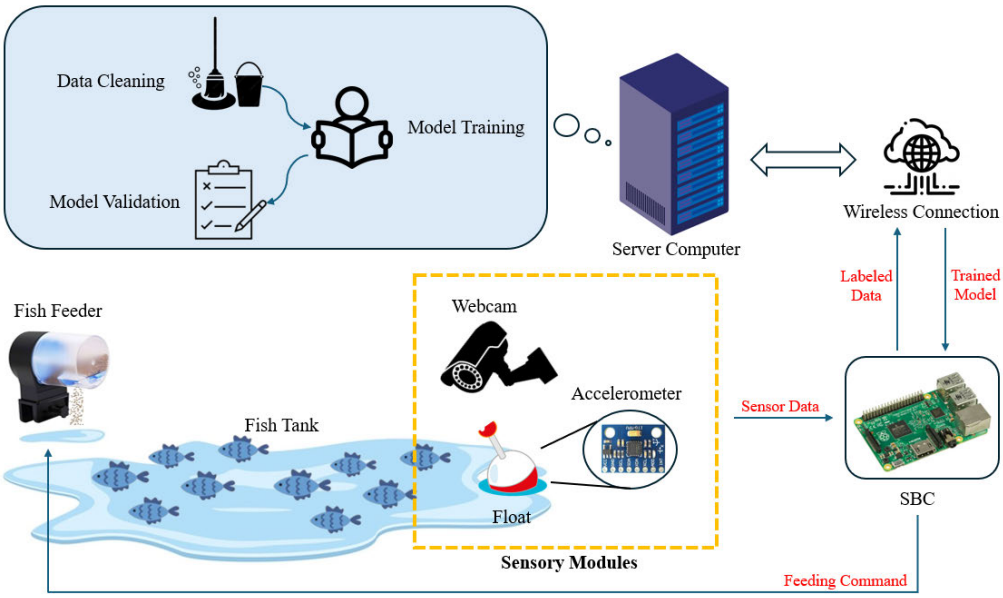


FIGURE 16. Experiment setup.

TABLE 2. Accelerometer signal models architecture: ANN (left), GRU (mid), 1D-CNN (right).

Layer	Input Dim.	Output Dim.
Input	20*3	-
Flatten	20*3	60
Dense	60	8
Output	8	8

Layer	Input Dim.	Output Dim.
Input	20*3	-
GRU	20*3	8
Output	8	8

Layer	Input Dim.	Output Dim.
Input	20*3	-
Conv1D 1	20*3	20*8
Conv1D 2	20*8	20*4
Conv1D 3	20*4	20*2
Output	20*2	8

blocks, and returns 2 values as output, representing binary class (high appetite and low appetite).

2) DETAIL PROPOSED MODEL

The point of the proposed method is to see the effect of introducing more trainable data, in addition to video data, on the model's accuracy. Here, the additional data are made as minimal as possible to avoid adding too many parameters to the model. To achieve this, we tried to add an accelerometer to the system. Figure 15 shows the comparison between the referenced model and the proposed model. We propose a multi-modal sensor by adding the accelerometer data.

The key idea of the accelerometer usage is simple. Right before a feeding process occurs, the fish will gather at the usual feeding spot. After the fish feed is given, the fish will go up to the surface, causing ripples and splashes. We can measure the water surface's movement by placing the accelerometer in a proper way near the feeding spot. This method was first introduced by the work in [19].

The data taken from the accelerometer are then pre-processed and fed into 1-dimensional models. We tried 3 different models to determine which one performs best for this specific case, i.e., the ANN model, the GRU-RNN model, and the 1-D CNN model. The models' architecture is shown in Table 2. The 1-dimensional models' outputs are then concatenated with the 3-dimensional (R(2+1)D) models as depicted in Figure 13.

TABLE 3. Models' parameters and inference times in various platforms (seconds).

Model	# Par.	Raspi	PC
Reference (R(2+1)D*)	1730082	4.657	0.115
R(2+1)D* + GRU	1785800	5.431	0.120
R(2+1)D* + Conv1D	1785706	4.689	0.121
R(2+1)D* + ANN	1785904	4.616	0.123

The model is able to be implemented in Raspberry Pi 4B. The computation takes around 4 to 5 seconds each. This is sufficient for real-time data processing since the feeding interval during an active feeding session can be controlled in the user's favor. Typically, the overall computation takes around 15 to 30 seconds for the fish (Nile Tilapia) that is used in the experiment. Finally, Table 3 shows the models' number of parameters and their performances (in seconds) in the Rasberry Pi 4 board and PC with Ryzen 9 Processor and 32GB memory.

TABLE 4. Used products' specifications.

Product	Specifications
Hikvision DS-U02	1920 × 1080 Resolution Min. illumination: 0.1 Lux Max. FPS: 30 at 1920 × 1080
MPU6050	measurement range: 2g 16g Max. sensitivity: 16,384 LSB/g Max. data rate: 1 kHz

V. EXPERIMENTAL RESULTS

A. EXPERIMENT SETUP

A proper setup for the experiment is necessary to gain reliable data from the sensors. These data then are used for training and validating the models.

An aquarium, with all its utilities, is placed inside the lab. A camera (Hikvision DS-U02), a floating accelerometer (MPU6050), and a fish feeder (self-made) are placed within the aquarium. Those 3 are connected by wire to an SBC (Raspberry Pi 4B). These products' specifications are listed in Table 4. The sensors (the camera and the accelerometer) pass the sensing data to the SBC, whereas the feeder feeds the fish based on the feeding command from the SBC. The sensing data then are labeled and sent to the server computer for further preprocessing, training, and validation. Figure 16 illustrates this setup. Note that the fish feeder is placed between the accelerometer and the camera so that it can provide similar information to both sensors. Figure 17 shows the implementation of the setup. The accelerometer is placed inside a water-resistant float as designed in Figure 18. The size of the floating accelerometer is $100 \times 100 \times 69.12$ cm^3.

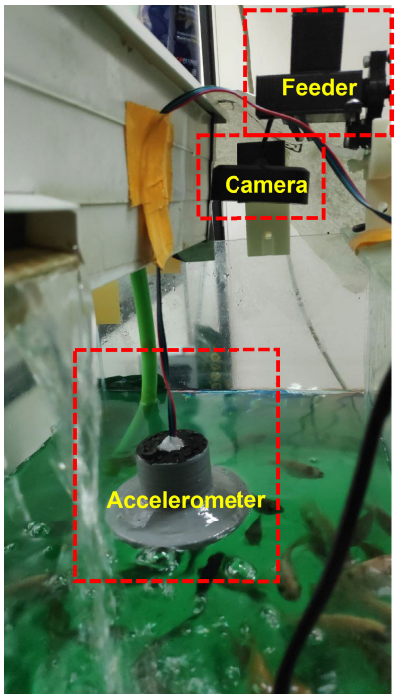


FIGURE 17. Implementation setup with arrangement inside the aquarium.

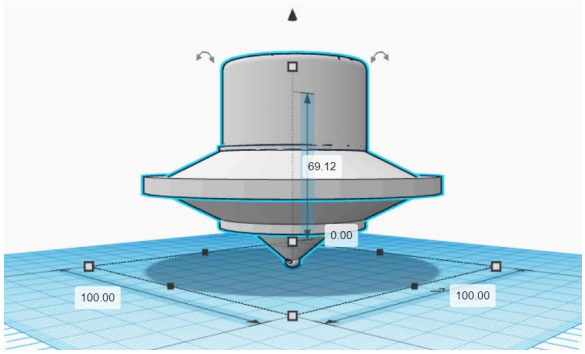


FIGURE 18. Design of floating accelerometer (dimensions in mm).

B. DATA COLLECTION

In general, we want to construct the models so that they can distinguish whether the fish need to be fed or not (high appetite or low appetite), and thus, the data needs to be taken to represent those two conditions.

The high appetite data are taken during the feeding processes. First, the fish are starved for some time to make sure that their appetite will increase significantly (at least 8 hours). After that, the feeding is done while the sensors record a few sets of data from the aquarium. Approaching the end of the process, we start to take the low appetite labeled data. Also, to add more data variety, part of the low appetite labeled data is taken without any feeding process occurring. This whole procedure can be seen in the flowchart in Figure 3. In the end, we were able to collect 2475 data, with 9.15% of them labeled as high appetite (see Table 5).

To match the information from the videos and the accelerometer signals, we program the SBC to immediately read the accelerometer after each frame reading is completed. This way, both data will have the same timestamp. In this case, both are set to have 20 sequences of 20-fps data stream.

TABLE 5. Data statistics with training-testing data segmentation.

Total Data 2475 (100%)	High Appetite 226 (9.15%)	Low Appetite 2249 (90.85%)
Test Data	113 (50%)	267 (11.89%)
Train Data	113 (50%)	1982 (88.11%)

C. DATASET CREATION

After the data are taken, we preprocess the data and create datasets for training and validation. This procedure is used to address a few defects inside these data so that unnecessary models' convergence can be avoided.

For the video data, we applied the glare removal technique to each frame. This is needed because of the lighting problem in the lab, causing several frames of the videos to possess high-intensity pixels. The glare removal is done by creating a mask for white-ish pixels with a certain threshold and then

blending those pixels with their neighbors. We also tried to generate more data by varying the frames' intensity. The result is shown in Figure 19.

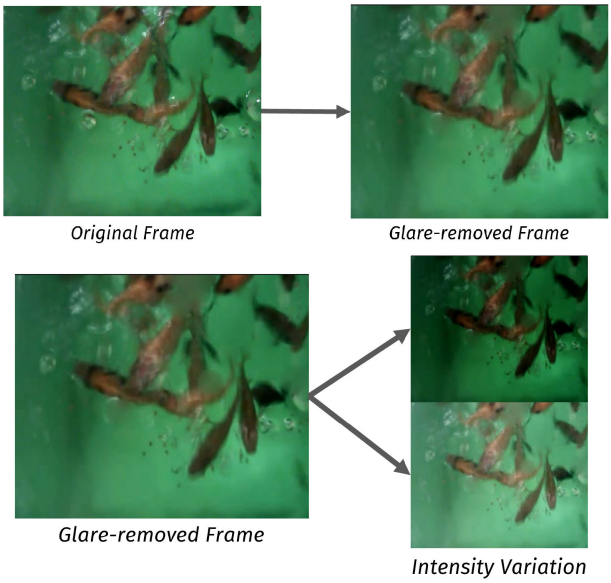


FIGURE 19. Glare removal and intensity varying results.

As for the acceleration data, we preprocessed them by removing the static component of every data using $y[k] = x[k] - \frac{\sum_{l=0}^{N-1} x[l]}{N}$ with y representing the result, x as the original data, and N as the number of data sequence.

Moreover, we created a different acceleration dataset by applying a high pass filter to the zero-mean data to attenuate lower frequencies. The zero-mean data is still needed for this case to ensure the first few data sequences do not explode.

The filtering frequencies themselves were designed by analyzing the water surface movement inside the aquarium during active feeding and idle conditions. From that, we obtained that idle and feeding conditions are characterized by frequencies around 4 Hz. Therefore, we tried to filter the data by still including the 4 Hz signals. Moreover, we varied the filter choices and their cutoff frequencies to determine the most suitable one. From the observation, we experiment with several cutoff frequencies that include 4 Hz. They are high pass filter (HPF) 0.25 Hz, HPF 0.5 Hz, low pass filter (LPF) 9.5 Hz, and LPF 9 Hz. These frequencies are the best performances that make the proposed architectures learn. Finally, these data are combined into two datasets (training dataset and validation dataset), divided as depicted in Table 5.

D. FILTERED MODEL RESULTS

From several experiments in varying the cutoff frequencies of the filter, we summarize the best performances in Table 6. Most of the information from the accelerator data is contained in the Zeromean models. From the Zeromean models, we can see that the Dense model outperforms the others with a maximum validation accuracy of 99.09%. On the other hand, the Conv1D model could not learn well with a maximum validation accuracy of only 79.37%.

TABLE 6. Preprocessing filter variation comparisons.

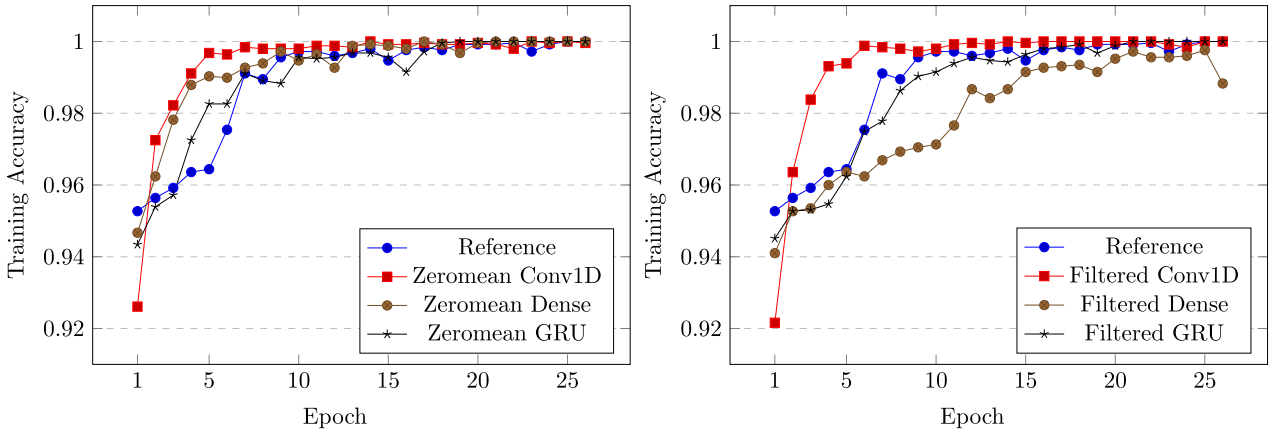
Filter	Cutoff Freq.	1D-Model	Max. Val. Acc.
Reference [22] (no 1D-model and no filter)			0.9569
Zeromean	-	Dense	**0.9909**
		GRU	0.9841
		Conv1D	0.7937
HPF	0.25 Hz	Dense	0.9598
		GRU	0.9887
		Conv1D	0.9502
HPF	0.5 Hz	Dense	0.9841
		GRU	0.9728
		Conv1D	0.9342
LPF	9.5 Hz	Dense	**0.9932**
		GRU	0.9841
		Conv1D	0.9728
LPF	9 Hz	Dense	0.9637
		GRU	0.9615
		Conv1D	0.8844

Next, we implemented four types of filters *i.e.,* HPF 0.25 Hz, HPF 0.5 Hz, LPF 9.5 Hz, and LPF 9 Hz. From this, we can see that the Conv1D models can learn well with the highest validation accuracy in the LPF 9.5 Hz filter (97.28%). If we see the nature of Conv1D in Lemma 11, the sum of weighted inputs, in this case several serial data of accelerometer, influences the results of the activation functions. By filtering the inputs, the extreme fluctuations in the data are reduced. Finally, the best performance from the filtered models is HPF 9.5 Hz with the dense network in the 1D model. The dense network is generic for approximating a function, thus by using enough neurons (in this case 60 neurons), the network can predict with a very good performance (99.32%).

E. TRAINING THE MODELS

Three models are trained using two datasets. Here, we obtained six results, apart from the reference model's result. The models are Zeromean GRU, Zeromean Conv1D, Filtered Dense (ANN), Filtered GRU, and Filtered Conv1D. The architectures of Dense (ANN), GRU, and Conv1D correspond to Tabel 1. We trained the models with 26 epochs each with the same parameters (Adamax with 0.001 learning rate, 32 data for each batch). These parameters were initially tuned for the adjusted reference model.

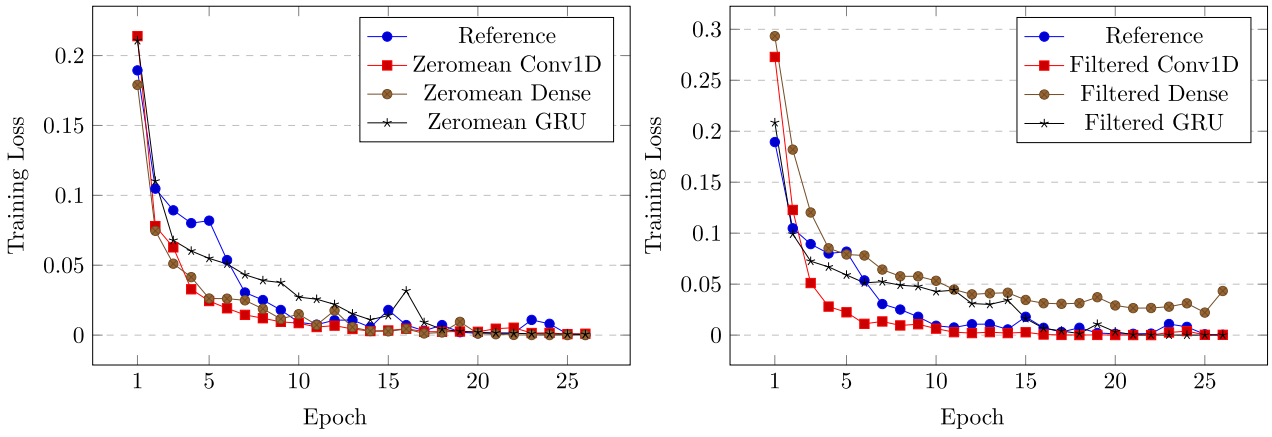
The training results can be seen in Figures 20 and 21 for the accuracy and loss statistics, respectively. It should be emphasized that we have way more low appetite labeled data than high ones, and thus the seemingly easy-to-converge training process does not represent its ease (for example, assuming the models only generate 'low appetite' results, the accuracy would still be above 90%). The plots indicate that there are no significant differences for every model during the training procedure. All of the models converge in a quite similar manner. However, several variations, such as the model using Conv1D, tend to have a better overall



(a) Training Accuracy for Zeromean Models

(b) Training Accuracy for Filtered Models

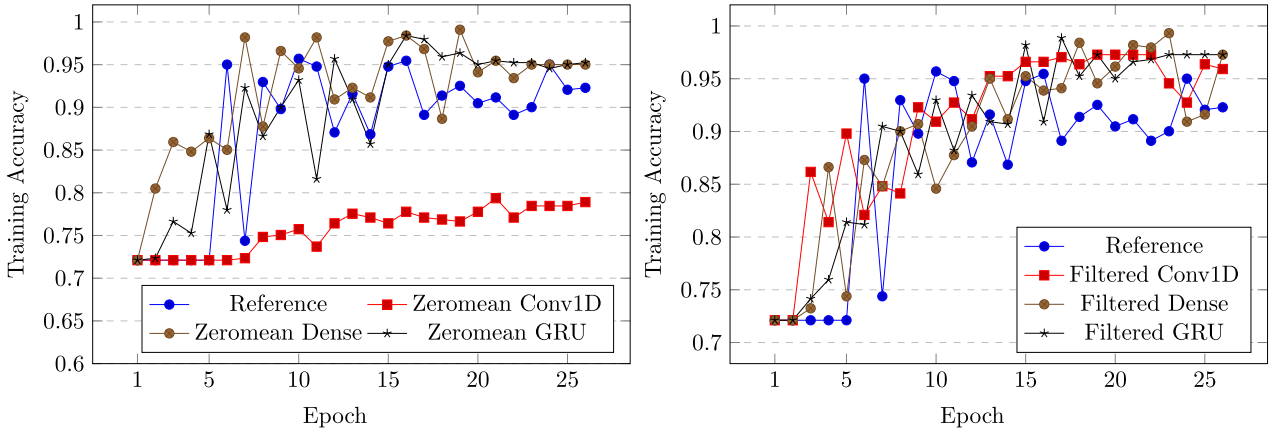
FIGURE 20. Training accuracy comparison for various methods.



(a) Training for Loss Zeromean Models

(b) Training for Loss Filtered Models

FIGURE 21. Training loss comparison for various methods.



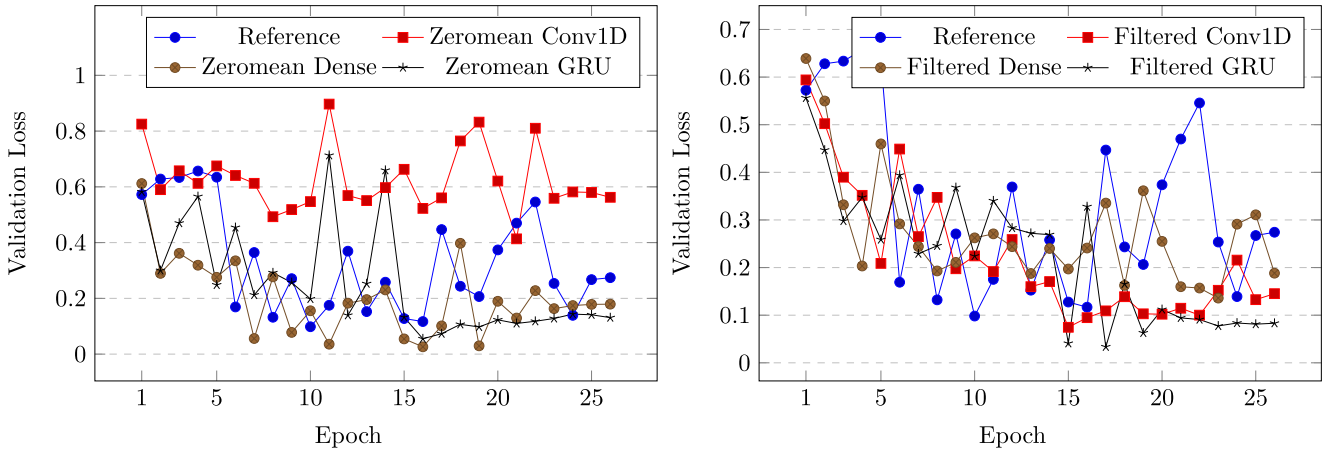
(a) Validation Accuracy for Zeromean Models

(b) Validation Accuracy for Filtered Models

FIGURE 22. Validation accuracy comparison for various methods.

performance than the referenced model (lower in loss, higher in accuracy).

Moreover, in the Zeromean models, the learning curves tend to saturate faster than those of the Filtered models. This



(a) Validation Loss for Zeromean Models

(b) Validation Loss for Filtered Models

FIGURE 23. Validation loss comparison for various methods.

is caused by no information being removed in the Zeromean models (no filter applied). It can be seen also, that in the epoch below 7, the proposed multi-modal models learn faster than the reference model. This is because of the additional information from the accelerometer data.

F. VALIDATION RESULT

During the training process, each epoch ended with a validation procedure. The models are used to make inferences to a separate dataset to see their reliability. Figures 22 and 23 show the validation accuracy and loss statistics for every model combination. In general, Figures 22b and 23b show that filtering the accelerometer data before the training process quite increases the models' accuracy as summarized in Table 6. It is also quite clear that the Conv1D cannot learn well in the Zeromean model since the nature of the model as mentioned before (Lemma 11). The highest validation level is achieved using the Dense model with filtered accelerometer data (99,32%, 3,63% improvement compared to the reference model with 95,69%).

Several models tend to overfit after 20 epochs. This can be seen by analyzing the correlation between Figure 20 and Figure 22. After the 20 epochs, few models gain nearly 100% training accuracy with almost 0 loss. Moreover, the validation accuracy is stuck in certain values. Therefore, the optimum epoch value is taken below 20.

In summary, the proposed hybrid models improve the reference model up to 99.09% for the Zeromean model and up to 99.32% for the Filtered model. Both best performances are obtained by the hybrid R(2+1)D and dense network model.

VI. CONCLUSION

In this paper, we addressed the problem of a fish feeder using artificial intelligence in an aquarium. We proposed a fish appetite detection using multi-modal sensors resulting in the data for our AI system. Our AI system consisted of R(2+1)D convolutional layers and dense networks to process video and accelerometer data. The video data was split into 20 frames and processed by R(2+1)D convolutional neural networks. Moreover, the accelerometer data was used to train several networks such as 1-dimensional CNN, GRU, and dense networks (ANN). The system was implemented in an aquarium with two sensors *i.e.,* a webcam camera and an accelerometer and a main board processing using Raspberry-Pi 4. Experimental results showed that the proposed system outperforms other methods with validation accuracy up to 99.09% for the Zeromean dense model and up to 99.39% for the Filtered dense model. The work is useful for automation and efficiency in aquacultures. Future works include applying more sensor nodes for the system and developing specialized hardware for the inference using FPGAs and application-specific ICs.

ACKNOWLEDGMENT

An earlier version of this paper was presented in part at the 2023 International Conference on Electrical Engineering and Informatics (2023 ICEEI), Bandung, Indonesia [DOI: 10.1109/ICEEI59426.2023.10346722] and in part at the Second International Conference on Maritime IT Convergence (2023 ICMIC), Jeju, South Korea [24].

REFERENCES

[1] (2022). *The State of World Fisheries and Aquaculture 2022–Towards Blue Transformation*. Accessed: Jun. 30, 2023.

[2] (2023). *OECD/FAO Agricultural Outlook 2023-2032*. [Online]. Available: https://www.oecd.org/en/publications/2023/07/oecd-fao-agricultural-outlook-2023-2032_859ba0c2.html

[3] C. Wang, Z. Li, T. Wang, X. Xu, X. Zhang, and D. Li, "Intelligent fish farm—The future of aquaculture," *Aquaculture Int.*, vol. 29, no. 6, pp. 2681–2711, Dec. 2021.

[4] L.-B. Chen, Y.-H. Liu, X.-R. Huang, W.-H. Chen, and W.-C. Wang, "Design and implementation of a smart seawater aquarium system based on artificial intelligence of things technology," *IEEE Sensors J.*, vol. 22, no. 20, pp. 19908–19918, Oct. 2022.

[5] J. Voas, B. Agresti, and P. A. Laplante, "A closer look at IoT 's things," *IT Prof.*, vol. 20, no. 3, pp. 11–14, May 2018.

[6] C. N. Udanor, N. I. Ossai, E. O. Nweke, B. O. Ogbuokiri, A. H. Eneh, C. H. Ugwuishiwu, S. O. Aneke, A. O. Ezuwgu, P. O. Ugwoke, and A. Christiana, "An Internet of Things labelled dataset for aquaponics fish pond water quality monitoring system," *Data Brief*, vol. 43, Aug. 2022, Art. no. 108400.
[7] H. Yu, Z. Wang, H. Qin, and Y. Chen, "An automatic detection and counting method for fish lateral line scales of underwater fish based on improved yolov5," *IEEE Access*, vol. 11, pp. 143616–143627, 2023.
[8] H. Pradana and K. Horio, "Automatic controlling fish feeding machine using feature extraction of nutriment and ripple behavior," *Int. J. Innov. Comput., Inf. Control*, vol. 17, pp. 1–19, Apr. 2021.
[9] J. Zhao, W. J. Bao, F. D. Zhang, Z. Y. Ye, Y. Liu, M. W. Shen, and S. M. Zhu, "Assessing appetite of the swimming fish based on spontaneous collective behaviors in a recirculating aquaculture system," *Aquacultural Eng.*, vol. 78, pp. 196–204, Aug. 2017.
[10] L. Shu and X. Wen, "An aquaculture monitoring system based on NB-IoT," in *Proc. 33rd Chin. Control Decis. Conf. (CCDC)*, May 2021, pp. 5620–5624.
[11] M. Lafont, S. Dupont, P. Cousin, A. Vallauri, and C. Dupont, "Back to the future: IoT to improve aquaculture : Real-time monitoring and algorithmic prediction of water parameters for aquaculture needs," in *Proc. Global IoT Summit (GIoTS)*, Jun. 2019, pp. 1–6.
[12] J. Huang, J. Wan, J. Yu, F. Zhu, and Y. Ren, "Edge computing-based adaptable trajectory transmission policy for vessels monitoring systems of marine fishery," *IEEE Access*, vol. 8, pp. 50684–50695, 2020.
[13] M. A. M. Razman, G. A. Susto, A. Cenedese, A. P. P. Abdul Majeed, R. M. Musa, A. S. Abdul Ghani, F. A. Adnan, K. M. Ismail, Z. Taha, and Y. Mukai, "Hunger classification of lates calcarifer by means of an automated feeder and image processing," *Comput. Electron. Agricult.*, vol. 163, Aug. 2019, Art. no. 104883.
[14] T.-H. Wu, Y.-I. Huang, and J.-M. Chen, "Development of an adaptive neural-based fuzzy inference system for feeding decision-making assessment in silver perch (Bidyanus bidyanus) culture," *Aquacultural Eng.*, vol. 66, pp. 41–51, May 2015.
[15] N. Ubina, S.-C. Cheng, C.-C. Chang, and H.-Y. Chen, "Evaluating fish feeding intensity in aquaculture with convolutional neural networks," *Aquacultural Eng.*, vol. 94, Aug. 2021, Art. no. 102178.
[16] M. A. Adegboye, A. M. Aibinu, J. G. Kolo, I. Aliyu, T. A. Folorunso, and S.-H. Lee, "Incorporating intelligence in fish feeding system for dispensing feed based on fish feeding intensity," *IEEE Access*, vol. 8, pp. 91948–91960, 2020.
[17] D. Li, Z. Wang, S. Wu, Z. Miao, L. Du, and Y. Duan, "Automatic recognition methods of fish feeding behavior in aquaculture: A review," *Aquaculture*, vol. 528, Nov. 2020, Art. no. 735508.
[18] C. M. Chang, W. Fang, R. C. Jao, C. Z. Shyu, and I. C. Liao, "Development of an intelligent feeding controller for indoor intensive culturing of eel," *Aquacultural Eng.*, vol. 32, no. 2, pp. 343–353, Jan. 2005.
[19] A. Subakti, Z. F. Khotimah, and F. M. Darozat, "Preliminary study of acceleration based sensor to record Nile tilapia (Oreochromis niloticus) feeding behavior at water surface," *J. Phys., Conf. Ser.*, vol. 795, Jan. 2017, Art. no. 012060, doi: 10.1088/1742-6596/795/1/012060.
[20] S. Zhao, W. Ding, S. Zhao, and J. Gu, "Adaptive neural fuzzy inference system for feeding decision-making of grass carp (Ctenopharyngodon idellus) in outdoor intensive culturing ponds," *Aquaculture*, vol. 498, pp. 28–36, Jan. 2019.
[21] D. Wei, F. Zhang, Z. Ye, S. Zhu, D. Ji, J. Zhao, F. Zhou, and X. Ding, "Effects of intelligent feeding method on the growth, immunity and stress of juvenile micropterus salmoides," *Artif. Intell. Agricult.*, vol. 5, pp. 118–124, May 2021.
[22] W.-C. Hu, L.-B. Chen, B.-K. Huang, and H.-M. Lin, "A computer vision-based intelligent fish feeding system using deep learning techniques for aquaculture," *IEEE Sensors J.*, vol. 22, no. 7, pp. 7185–7194, Apr. 2022.
[23] A. D'sky, I. Syafalni, N. Sutisna, H. Hariyanto, T. Adiono, and E. Sumiarsih, "A novel multi-modal deep learning method for Fish's appetite detection," in *Proc. Int. Conf. Electr. Eng. Informat. (ICEEI)*, Oct. 2023, pp. 1–6.
[24] T. Adiono, A. D'sky, I. Syafalni, N. Sutisna, and H. Hariyanto, "Cnn and rnn-based deep learning methods for fish's appetite detection," in *Proc. 2nd Int. Conf. Maritime IT Converg.*, 2023, pp. 1–4.
[25] J.-H. Wang, S.-K. Lee, Y.-C. Lai, C.-C. Lin, T.-Y. Wang, Y.-R. Lin, T.-H. Hsu, C.-W. Huang, and C.-P. Chiang, "Anomalous behaviors detection for underwater fish using AI techniques," *IEEE Access*, vol. 8, pp. 224372–224382, 2020.
[26] J. Horie, T. Sasakura, J. Horie, Y. Ina, Y. Mashino, H. Mitamura, K. Moriya, T. Noda, N. Arai, H. Mitamura, and N. Arai, "Development of a pinger for classification of feeding behavior of fish based on axis-free acceleration data," in *Proc. Techno-Ocean (Techno-Ocean)*, Oct. 2016, pp. 268–271.
[27] J. Kristmundsson, Ø. Patursson, J. Potter, and Q. Xin, "Fish monitoring in aquaculture using multibeam echosounders and machine learning," *IEEE Access*, vol. 11, pp. 108306–108316, 2023.
[28] F. Han, J. Zhu, B. Liu, B. Zhang, and F. Xie, "Fish shoals behavior detection based on convolutional neural network and spatiotemporal information," *IEEE Access*, vol. 8, pp. 126907–126926, 2020.
[29] K. Terayama, K. Shin, K. Mizuno, and K. Tsuda, "Integration of sonar and optical camera images using deep neural network for fish monitoring," *Aquacultural Eng.*, vol. 86, Aug. 2019, Art. no. 102000.
[30] R. Schraml, H. Hofbauer, E. Jalilian, D. Bekkozhayeva, M. Saberioon, P. Cisar, and A. Uhl, "Towards fish individuality-based aquaculture," *IEEE Trans. Ind. Informat.*, vol. 17, no. 6, pp. 4356–4366, Jun. 2021.
[31] Y. Atoum, S. Srivastava, and X. Liu, "Automatic feeding control for dense aquaculture fish tanks," *IEEE Signal Process. Lett.*, vol. 22, no. 8, pp. 1089–1093, Aug. 2015.
[32] H. S. AlZubi, W. Al-Nuaimy, J. Buckley, and I. Young, "An intelligent behavior-based fish feeding system," in *Proc. 13th Int. Multi-Conf. Syst., Signals*, 2016, pp. 1–14.
[33] N. P. Pandit and M. Nakamura, "Effect of high temperature on survival, growth and feed conversion ratio of Nile tilapia, Oreochromis niloticus," *Our Nature*, vol. 8, no. 1, pp. 219–224, Jan. 1970.
[34] P. J. Ashley, "Fish welfare: Current issues in aquaculture," *Appl. Animal Behaviour Sci.*, vol. 104, nos. 3–4, pp. 199–235, May 2007.
[35] T. Hastein, A. D. Scarfe, and V. L. Lund, "Science-based assessment of welfare: Aquatic animals," in *Revue Scientifique Et Technique-Office International Des Epizooties*. Paris, France: OIE Office International Des Epizooties, 2005.
[36] E. Carter, "FAWC report considers the welfare of farmed fish," *Animal Welfare*, vol. 23, no. 2, pp. 232–233, May 2014.
[37] K. M. Pai, K. B. A. Shenoy, and M. M. M. Pai, "A computer vision based behavioral study and fish counting in a controlled environment," *IEEE Access*, vol. 10, pp. 87778–87786, 2022.
[38] Y. Zhou, H. Yu, J. Wu, Z. Cui, H. Pang, and F. Zhang, "Fish density estimation with multi-scale context enhanced convolutional neural network," *J. Commun. Inf. Netw.*, vol. 4, no. 3, pp. 80–88, Sep. 2019.
[39] L. J. Nowak and M. Lankheet, "Fish detection using electrical impedance spectroscopy," *IEEE Sensors J.*, vol. 22, no. 21, pp. 20855–20865, Nov. 2022.
[40] J. Gladju, B. S. Kamalam, and A. Kanagaraj, "Applications of data mining and machine learning framework in aquaculture and fisheries: A review," *Smart Agricult. Technol.*, vol. 2, Dec. 2022, Art. no. 100061.
[41] S. Zhao, S. Zhang, J. Liu, H. Wang, J. Zhu, D. Li, and R. Zhao, "Application of machine learning in intelligent fish aquaculture: A review," *Aquaculture*, vol. 540, Jul. 2021, Art. no. 736724.
[42] P. W. Khan and Y. C. Byun, "Optimized dissolved oxygen prediction using genetic algorithm and bagging ensemble learning for smart fish farm," *IEEE Sensors J.*, vol. 23, no. 13, pp. 15153–15164, Jul. 2023.
[43] A. Nawrocka, M. Nawrocki, and A. Kot, "Analysis of the biological (EMG) signal," in *Proc. 21st Int. Carpathian Control Conf. (ICCC)*, Oct. 2020, pp. 1–4.
[44] Y. Yang, J. Lu, B. D. Pflugrath, H. Li, J. J. Martinez, S. Regmi, B. Wu, J. Xiao, and Z. D. Deng, "Lab-on-a-fish: Wireless, miniaturized, fully integrated, implantable biotelemetric tag for real-time in vivo monitoring of aquatic animals," *IEEE Internet Things J.*, vol. 9, no. 13, pp. 10751–10762, Jul. 2022.
[45] R. R. Harrison, H. Fotowat, R. Chan, R. J. Kier, R. Olberg, A. Leonardo, and F. Gabbiani, "Wireless neural/EMG telemetry systems for small freely moving animals," *IEEE Trans. Biomed. Circuits Syst.*, vol. 5, no. 2, pp. 103–111, Apr. 2011.
[46] K. Adhikari and S. Kay, "An exact solution for sparse sampling for optimal detection of known signals in Gaussian noise," *IEEE Signal Process. Lett.*, vol. 30, pp. 369–373, 2023.
[47] L. Wang, Q. Wang, J. Wang, and X. Zhang, "Fast band-limited sparse signal reconstruction algorithms for big data processing," *IEEE Sensors J.*, vol. 23, no. 12, pp. 13084–13099, Jun. 2023.
[48] Y. Qin, M. A. Kishk, and M.-S. Alouini, "Stochastic-geometry-based analysis of multipurpose UAVs for package and data delivery," *IEEE Internet Things J.*, vol. 10, no. 5, pp. 4664–4676, Mar. 2023.
[49] M. Wang, C. Xu, X. Chen, H. Hao, L. Zhong, and D. O. Wu, "Design of multipath transmission control for information-centric Internet of Things: A distributed stochastic optimization framework," *IEEE Internet Things J.*, vol. 6, no. 6, pp. 9475–9488, Dec. 2019.

[50] M. Taneja, N. Jalodia, and A. Davy, "Distributed decomposed data analytics in fog enabled IoT deployments," *IEEE Access*, vol. 7, pp. 40969–40981, 2019.
[51] R. Soltanzadeh, B. Hardy, R. D. Mcleod, and M. R. Friesen, "A prototype system for real-time monitoring of Arctic char in indoor aquaculture operations: Possibilities & challenges," *IEEE Access*, vol. 8, pp. 180815–180824, 2020.
[52] C. Hu, B. Murch, A. A. Corcoran, L. Zheng, B. B. Barnes, R. H. Weisberg, K. Atwood, and J. M. Lenes, "Developing a smart semantic web with linked data and models for near-real-time monitoring of red tides in the eastern Gulf of Mexico," *IEEE Syst. J.*, vol. 10, no. 3, pp. 1282–1290, Sep. 2016.
[53] F. Yuan, Y. Huang, X. Chen, and E. Cheng, "A biological sensor system using computer vision for water quality monitoring," *IEEE Access*, vol. 6, pp. 61535–61546, 2018.
[54] M. K. Moghimi and F. Mohanna, "Reliable object recognition using deep transfer learning for marine transportation systems with underwater surveillance," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2515–2524, Feb. 2023.
[55] D. Liu, B. Xu, Y. Cheng, H. Chen, Y. Dou, H. Bi, and Y. Zhao, "Shrimpseed_Net: Counting of shrimp seed using deep learning on smartphones for aquaculture," *IEEE Access*, vol. 11, pp. 85441–85450, 2023.
[56] C. C. Aggarwal, L.-F. Aggarwal, and Lagerstrom-Fife, *Linear Algebra and Optimization for Machine Learning*, vol. 156. Cham, Switzerland: Springer, 2020.
[57] I. Goodfellow, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
[58] A. D'sky, I. Syafalni, N. Sutisna, H. Hariyanto, T. Adiono, and E. Sumiarsih, "A novel multi-modal deep learning method for fish's appetite detection," in *Proc. Int. Conf. Elect. Eng. Inform. (ICEEI)*, Bandung, Indonesia, 2023, pp. 1–6, doi: 10.1109/ICEEI59426.2023.10346722.

INFALL SYAFALNI (Member, IEEE) received the B.Eng. degree in electrical engineering from Institut Teknologi Bandung (ITB), Bandung, Indonesia, in 2008, the M.Sc. degree in electronics engineering from the University of Science Malaysia (USM), Penang, Malaysia, in 2011, and the Dr.Eng. degree in engineering from the Kyushu Institute of Technology (KIT), Iizuka, Japan, in 2014. From 2014 to 2015, he held a research position with KIT. From 2015 to 2018, he was an ASIC Engineer with the ASIC Development Group, Logic Research Company Ltd., Fukuoka, Japan. In 2019, he joined ITB, where he is currently an Assistant Professor with the School of Electrical Engineering and Informatics and a Researcher with the University Center of Excellence on Microelectronics. In 2023, he was a Visiting Scholar with the System Technology Co-Optimization (STCO) Program, Interuniversity Microelectronics Centre (IMEC), Leuven, Belgium. In 2024, he was a Visiting Scholar with the Systems Design Laboratory, The University of Tokyo, Tokyo, Japan. His research interests include logic synthesis, logic design, VLSI design, and efficient circuits and algorithms.

AGAPE D'SKY (Graduate Student Member, IEEE) received the bachelor's degree in electrical engineering and the master's degree in microelectronics from Bandung Institute of Technology in 2023 and 2024, respectively. His previous works and projects include robotics, embedded systems, AI, and RTL design, focusing on integrating hardware and software.

NANA SUTISNA (Member, IEEE) received the B.S. degree in electrical engineering and the M.S. degree in microelectronics from Bandung Institute of Technology, Indonesia, in 2005 and 2011, respectively, and the Ph.D. degree in computer science and electronics from Kyushu Institute of Technology, in 2017. From 2017 to 2020, he was a Postdoctoral Fellow with the Department of Computer Science and System Engineering, Kyushu Institute of Technology. In 2019, he joined Institut Teknologi Bandung as an Assistant Professor. From October to December 2023, he was with IMEC Belgium as a Visiting Researcher, working on an RL-based intelligent hybrid memory management system. His research interests include VLSI design, baseband wireless system design, AI processor design, and HW/SW co-design and co-verification.

TRIO ADIONO (Senior Member, IEEE) received the B.Eng. degree in electrical engineering and the M.Eng. degree in microelectronics from the Institut Teknologi Bandung, Indonesia, in 1994 and 1996, respectively, and the Ph.D. degree in VLSI design from Tokyo Institute of Technology, Japan, in 2002. He is currently a Professor with the School of Electrical Engineering and Informatics and also the Head of the IC Design Laboratory, Microelectronics Center, Institut Teknologi Bandung. He holds a Japanese patent on a high-quality video compression systems. His research interests include VLSI design, signal and image processing, VLC, smart cards, and electronics solution design and integration.